```cpp
#include<iostream>
using namespace std;

struct Node
{
	float marks = 0;
	Node* next = nullptr;
	Node* prev = nullptr;
};

class DLL
{
	long list_size = 0;
	Node* starting_node = nullptr;
	Node* ending_node = nullptr;

public:
	DLL(void) {}
	~DLL(void);
	void print(void);
	void push_front(float marks_to_add);
	void push_back(float marks_to_add);
	void delete_kth(long index);
	float at(long index);
	void swap_all(void);
};

void DLL::swap_all(void)
{
	if (list_size < 2) {
		return;
	}
	Node* current_node = starting_node;
	while (current_node != nullptr && current_node->next != nullptr) {
		Node* temp_next = current_node->next;
		Node* temp_prev = current_node->prev;

		current_node->next = temp_next->next;
		current_node->prev = temp_next;

		temp_next->next = current_node;
		temp_next->prev = temp_prev;

		if (temp_prev != nullptr) {
			temp_prev->next = temp_next;
		}
		else {
			starting_node = temp_next;
		}
		if (current_node->next != nullptr) {
			current_node->next->prev = current_node;
		}
		else {
			ending_node = current_node;
		}
		current_node = current_node->next;
	}
}
```

```
float DLL::at(long index)
{
	Node* current_node = starting_node;
	for (long i = 1; i < index; i++)
	{
		current_node = current_node->next;
	}
	return current_node->marks;
}

void DLL::delete_kth(long index)
{
	if (index == 1)
	{
		Node* node_to_delete = starting_node;
		starting_node = starting_node->next;
		if (starting_node != nullptr) {
			starting_node->prev = nullptr;
		}
		delete node_to_delete;
	}
	else if (index == list_size)
	{
		Node* node_to_delete = ending_node;
		ending_node = ending_node->prev;
		if (ending_node != nullptr) {
			ending_node->next = nullptr;
		}
		delete node_to_delete;
	}
	else
	{
		Node* current_node = starting_node;
		for (long i = 1; i < index; i++)
		{
			current_node = current_node->next;
		}
		Node* node_to_delete = current_node;
		current_node->prev->next = current_node->next;
		current_node->next->prev = current_node->prev;
		delete node_to_delete;
	}
	list_size--;
}

void DLL::push_back(float marks_to_add)
{
	cout << " Adding to back: " << marks_to_add << endl;
	Node* new_node = new Node;
	new_node->marks = marks_to_add;
	new_node->next = nullptr;
	if (list_size == 0)
	{
		starting_node = new_node;
		ending_node = new_node;
	}
```

```
	else
	{
		new_node->prev = ending_node;
		ending_node->next = new_node;
		ending_node = new_node;
	}
	list_size++;
}

DLL::~DLL(void)
{
	Node* current_node = starting_node;
	for (; current_node != nullptr;)
	{
		Node* next_node = current_node->next;
		cout << " Deleting: " << current_node->marks << endl;
		delete current_node;
		current_node = next_node;
	}
}

void DLL::print(void)
{
	cout << endl << " Printing list...." << endl;
	if (list_size == 0)
	{
		cout << " The list is empty!!" << endl;
	}
	else
	{
		Node* current_node = starting_node;
		for (long i = 1; i <= list_size; i++)
		{
			if (current_node != nullptr)
			{
				cout << " " << current_node->marks << ", ";
				current_node = current_node->next;
			}
		}
		cout << endl;
	}
	cout << endl;
}

void DLL::push_front(float marks_to_add)
{
	cout << " Adding to front: " << marks_to_add << endl;
	Node* new_node = new Node;
	new_node->marks = marks_to_add;
	new_node->next = starting_node;
	new_node->prev = nullptr;
	if (list_size == 0)
	{
		starting_node = new_node;
		ending_node = new_node;
	}
```

```
	else
	{
		starting_node->prev = new_node;
		starting_node = new_node;
	}
	list_size++;
}

int main()
{
	DLL L;
	L.print();
	L.push_front(50);
	L.push_front(40);
	L.push_front(30);
	L.push_front(20);
	L.push_front(10);
	L.push_back(60);
	L.push_back(70);
	L.push_back(80);
	L.push_back(90);
	L.push_back(100);
	L.print();
	cout << " Swapping" << endl;
	L.swap_all();
	L.print();
	cout << endl << " Marks at index 3: " << L.at(3) << endl << endl;
	cout << " Deleting marks at index 3: " << L.at(3) << endl;
	L.delete_kth(3);
	L.print();
	return 0;
}
```